LAB 10 — K-Means Clustering

Finding Customer Segments, Choosing K and Avoiding the Traps on Real Data

Course	Machine Learning — BS Computer Science, 5th Semester
Duration	3 hours (single session)
Main Dataset	Mall Customers (Kaggle) — 200 customers: Age, Gender, Annual Income, Spending Score. https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python (file: Mall_Customers.csv)
Second Dataset	Wine (UCI) — 178 wines, 13 chemical measurements, 3 true cultivars. Built into scikit-learn: from sklearn.datasets import load_wine. Source: https://archive.ics.uci.edu/dataset/109/wine
Prior Knowledge	K-Means lecture (algorithm, distance, inertia, elbow, silhouette); StandardScaler (Labs 6 and 8); PCA plot (Lab 8)
Deliverable	One completed Colab/Jupyter notebook (see Submission Requirements)

Learning Objectives

- Turn the lecture's pen-and-paper K-Means examples into code and check every number against the slides.
- Write your own K-Means function and prove it matches scikit-learn.
- Choose K with the elbow method and the silhouette score, and calculate a silhouette by hand.
- Measure how much the starting points change the answer (local minima) and use k-means++ and n_init correctly.
- Show with experiments that feature units and scaling decide the clusters.
- See what one outlier does to K-Means and detect it with a distance rule.
- Turn clusters into named customer segments with a business action for each.

Before You Start

- You have already studied K-Means in the lecture: K, centroid, distance, assign/update, inertia, elbow and silhouette. This lab does not teach it again. It makes you use it, test it and question it.
- There is no target column, so there is no train/test split and no accuracy. Every result must be judged by distances, silhouette, plots and common sense.
- Each task gives an objective and steps, not full code. A 'Key definitions' table at the start of every Part reminds you of the words and functions you need.
- Use SEED = 42 everywhere, and write a short interpretation under every table and plot. A plot without an explanation earns no marks.

Part 0 — Setup and First Look (10 minutes)

Today there is no target column. The Mall Customers data has no 'right answer' for each customer, so we do not split into train and test and we cannot use accuracy. You will find groups with K-Means (your lecture) and judge them by distance, silhouette and common sense.

Key definitions

Term	What it means	How you write / read it
Unsupervised learning	Learning from data that has no labels. The goal is to find structure (groups), not to predict a column.	No y, no train_test_split.
Cluster	A group of points that are close to each other and far from the other groups.	model.labels_ gives the cluster number of every row.
Feature	One column used to describe each row (here Age, Income, Spending).	df[['Income','Spending']].values

Task 0.1 — Load and check the data

Load Mall_Customers.csv and understand it in one cell.

1. Load the CSV. Rename 'Annual Income (k$)' to Income and 'Spending Score (1-100)' to Spending (shorter names).
2. Print the shape, the number of missing values, the number of duplicate rows and the count of each Gender.
3. Show describe() for Age, Income and Spending.

Task 0.2 — Look at the data before clustering

Use your eyes first: draw three scatter plots side by side.

1. Draw Spending vs Income, Spending vs Age and Income vs Age (one row of three plots, axis labels and titles).

Questions to answer in Markdown (use your own numbers)

(a) How many groups can you see by eye in each of the three plots? Which plot shows the groups most clearly?

(b) Why can we not use accuracy or a train/test split to judge a clustering result?

Part 1 — K-Means From Scratch (35 minutes)

In the lecture you did K-Means by hand. Here you turn your pen-and-paper steps into code, check your answers against the lecture slides, and then build your own K-Means function and compare it with scikit-learn. Using KMeans() inside your own function is not allowed.

Key definitions

Term	What it means	How you write / read it
K	The number of clusters. You choose it before running.	KMeans(n_clusters=5)
Centroid	The centre (mean) of a cluster. It is a point, not always a real row.	model.cluster_centers_
Distance	1 feature: |x - m|. 2 or more features: Euclidean, sqrt of the sum of squared differences.	np.abs(a-b) or np.sqrt(((a-b)**2).sum(axis=1))

Term	What it means	How you write / read it
Assign step	Give each point to its nearest centroid.	d.argmin(axis=1)
Update step	Move each centroid to the mean of the points assigned to it.	X[labels==k].mean(axis=0)
Convergence	Stop when no point changes cluster (centroids stop moving).	np.allclose(new_c, c)
Inertia	Sum of squared distances from every point to its own centroid. Smaller = tighter clusters.	model.inertia_
Broadcasting	NumPy trick: x[:, None] - m[None, :] gives every point-to-centroid difference in one line.	x[:, None, :] - C[None, :, :]

Task 1.1 — Lecture Example 1 in code (one feature)

Reproduce the 9-number example from the lecture (2, 4, 10, 12, 3, 20, 30, 11, 25) with K = 2 and starting centroids 4 and 11.

1. Write a loop that, in every iteration, computes the distances, assigns the clusters, updates the centroids and prints both clusters and the new centroids.
2. Stop when no centroid changes. Print the final centroids and the inertia.
3. Compare with the lecture slides (final centroids 7 and 25, inertia 150).

Hints

- Distance matrix: np.abs(x[:, None] - m[None, :]) has shape (9, 2).
- Cluster of each point: dist.argmin(axis=1).

Task 1.2 — Lecture Example 2 in code (two features)

Do the height/weight example: P1 (185, 72), P2 (170, 56), P3 (168, 60), P4 (179, 68), P5 (182, 72), P6 (188, 77). K = 2, start with P1 and P2.

1. Use Euclidean distance and the same loop idea. Print the cluster of every point and the centroids per iteration.
2. Print the final distance table and check one value by hand (the lecture shows d(P3, M2) = 4.47 in the first iteration).

Task 1.3 — Write your own K-Means function

Write my_kmeans(X, init_centroids, max_iter=100) for any number of features.

1. Return four things: labels, final centroids, inertia and the number of iterations.
2. Test it on Example 1: it must give centroids 7 and 25 and inertia 150.
3. Think about what to do if a cluster ends up empty (keep its old centroid).

Task 1.4 — Check your function against scikit-learn

Run your function and KMeans on the same data with the same starting centroids and compare.

1. Use X2 = Income and Spending (200 x 2). Take the first 5 rows as the starting centroids.
2. Run my_kmeans. Run KMeans(n_clusters=5, init=<your 5 rows>, n_init=1, tol=0, algorithm='lloyd').
3. Compare iterations, inertia, labels and centroids.

Questions to answer in Markdown (use your own numbers)

(a) Example 1: the point 10 started in cluster C2 but moved to C1 in iteration 2. Explain why, using the centroid values.

(b) What makes the loop stop? Why is the stopping guaranteed even though the final answer is not always the best one?

(c) Did your function match scikit-learn exactly (labels, centroids, inertia)? If anything differed, explain why.

(d) What does a centroid mean for a group of customers? Why must the features be numeric for K-Means?

Part 2 — Choosing K: Elbow and Silhouette (30 minutes)

The algorithm needs K before it starts, but nobody tells us K. The lecture gave two tools: the elbow (inertia) and the silhouette score. Here you use both, then calculate a silhouette by hand to prove you understand it.

Key definitions

Term	What it means	How you write / read it
Elbow method	Plot inertia for K = 2, 3, 4 ... Inertia always falls as K grows, so look for the K where the fall suddenly becomes small (the bend).	model.inertia_ in a loop over K
a	Average distance from a point to the other points in its own cluster (smaller = better).	d[same_cluster].mean()
b	Average distance from the point to the points of the nearest other cluster (larger = better).	min over other clusters
Silhouette s	s = (b - a) / max(a, b). Close to +1: well placed. Near 0: on the border. Negative: probably in the wrong cluster.	silhouette_samples(X, labels)
Average silhouette	The mean of s over all points. The K with the highest value is a good candidate.	silhouette_score(X, labels)

Task 2.1 — Elbow and silhouette for K = 2 to 10

Decide how many customer groups exist using Income and Spending (X2).

1. For K = 2 to 10 fit KMeans(n_clusters=K, n_init=10, random_state=SEED) and record the inertia and the average silhouette.
2. Show a table (K, Inertia, Silhouette) and two plots: elbow curve and silhouette bars.
3. Print the K with the highest silhouette.

Task 2.2 — Silhouette by hand

Prove that you understand the formula by calculating it yourself.

1. Fit K = 5. Write a function that returns a, b and s for one point (use only NumPy distances). Apply it to customer 1 (row 0) and compare with silhouette_samples.
2. Show the count, mean and minimum silhouette of every cluster, and the number of customers with a negative silhouette.
3. Draw a silhouette plot of all customers sorted inside each cluster.

Task 2.3 — The lecture practice problem

Points 1, 3, 5, 7, 11, 13, 15 with K = 2 and starting centroids 1 and 11 (the 'Practice' slide).

1. Run your my_kmeans on these points and print the final clusters, centroids and iterations.
2. Calculate a, b and s for the points 1, 7 and 11 with your function and compare with silhouette_samples.
3. Also write the iterations on paper (clusters and centroids after each iteration) and keep the photo or notes for your own revision.

Questions to answer in Markdown (use your own numbers)

(a) Which K does the elbow suggest and which K does the silhouette suggest? Do they agree? Quote the numbers (for example how much the inertia drops from K=4 to 5 and from 5 to 6).

(b) What is the silhouette of customer 1 and what does it tell you? Which cluster has the weakest average silhouette and how many customers have a negative value?

(c) In the practice problem, is point 1 or point 7 less well placed? Explain using a and b.

(d) Why should we never choose K only by the lowest inertia?

Part 3 — Starting Points and Stability (20 minutes)

The lecture said 'starting points matter - run several starts and keep the best'. You will now measure how much, using the same data and the same K.

Key definitions

Term	What it means	How you write / read it
Local minimum	A solution that cannot be improved by the assign/update loop but is not the best possible one.	A higher final inertia than the best run.
init='random'	Pick K random rows as the starting centroids.	KMeans(init='random')
init='k-means++'	Pick starting centroids that are far from each other. This is the default and is usually better.	KMeans(init='k-means++')
n_init	How many different starts to try. scikit-learn keeps the run with the lowest inertia.	KMeans(n_init=10)
Adjusted Rand Index (ARI)	Agreement between two groupings. 1 = same groups, about 0 = no better than random. It ignores the cluster numbers (cluster 2 in one run can be cluster 4 in another).	adjusted_rand_score(a, b)

Task 3.1 — 30 different seeds

Measure how often K-Means finds the best answer.

1. With K = 5 on X2, run KMeans 30 times (random_state = 0 to 29) in three settings: init='random' with n_init=1, k-means++ with n_init=1, k-means++ with n_init=10.
2. Build a table: setting, number of distinct final inertia values (round to 1 decimal), lowest, highest, and the percentage of runs that reached the best inertia.
3. Draw one histogram of the final inertia values for the three settings.

Task 3.2 — Best run vs worst run

See what a bad local minimum looks like.

1. From the random-init runs take the best and the worst. Draw both clusterings with their centroids side by side.
2. Print their inertia and their ARI.

Questions to answer in Markdown (use your own numbers)

(a) How many different final inertia values did the random-start setting produce, and what percentage of its runs reached the best one? Compare with the other two settings.

(b) Why can the same data and the same K give different answers?

(c) How do k-means++ and n_init help? What did n_init=10 give in your table?

(d) Why do we use ARI to compare two clusterings instead of comparing the label numbers directly?

Part 4 — Scaling: Units Decide the Clusters (25 minutes)

The lecture warned: 'Rs 5000 would swamp 3 visits'. You will prove it twice: once by only changing the unit of Income, and once on the Wine data where the features have very different sizes and the true groups are known.

Key definitions

Term	What it means	How you write / read it
Feature scaling	Putting all features on a similar scale so that no feature wins the distance only because its numbers are bigger.	StandardScaler().fit_transform(X)
StandardScaler	z = (x - mean) / std. Every column gets mean 0 and standard deviation 1.	StandardScaler()
External check	Comparing clusters with known true groups. Possible only for teaching data like Wine. Real clustering problems have no labels.	adjusted_rand_score(y_true, labels)
PCA (Lab 8)	Reduces many features to 2 so we can draw the data. Used here only for the plot.	PCA(n_components=2)

Task 4.1 — Change the unit, change the clusters

Use Age and Income with K = 4. Income is in thousands of dollars (15 to 137). Make a second copy where Income is in dollars (multiply by 1000).

1. Cluster four versions: raw k$, raw dollars, scaled k$, scaled dollars (StandardScaler, n_init=10, random_state=SEED).
2. Print ARI between: raw k$ vs raw dollars, scaled k$ vs scaled dollars, raw dollars vs scaled dollars.
3. Draw Age vs Income (in k$) coloured by the clusters of raw k$, raw dollars and scaled.

Task 4.2 — Wine data: raw vs scaled

Load the Wine data with load_wine(): 178 wines, 13 chemical measurements, 3 true grape cultivars. The true cultivar is used ONLY to check the result, never to fit K-Means.

1. Print the three largest and the smallest standard deviation of the features.
2. Run KMeans(K=3, n_init=10, random_state=SEED) on the raw data and on the scaled data. For each report ARI against the true cultivar, silhouette score and cluster sizes.

Task 4.3 — Draw it

Compare the three views in one picture.

1. Use one PCA of the scaled data and draw three plots with the same axes: true cultivars, raw clusters, scaled clusters.

Questions to answer in Markdown (use your own numbers)

(a) What happened when only the unit of Income changed? Explain it with the distance formula (which feature controls the distance now?).

(b) On the Wine data which feature dominated the raw distance and how big is its standard deviation compared with the others? What were the ARI values raw vs scaled?

(c) The silhouette was higher on the raw Wine data although the clustering was worse. Explain why a silhouette can mislead here and why comparing it between raw and scaled data is not fair.

(d) In a real project with no labels, how will you decide whether to scale? Write your rule in one or two sentences.

Part 5 — Outliers and Anomalies (20 minutes)

The lecture said 'one extreme point drags a centroid'. You will add one extreme customer, see the damage, and then catch it with a distance rule (the anomaly detection idea from the lecture).

Key definitions

Term	What it means	How you write / read it
Outlier	A point far away from all the others. K-Means uses means and squared distances, so one outlier has a big effect.	A very large distance to the nearest centroid.
Distance to own centroid	How far a point is from the centre of its cluster. Large = the point does not fit any group well.	model.transform(X).min(axis=1)
mean + 3 std rule	Flag a point if its distance is larger than the mean distance plus 3 standard deviations.	d > d.mean() + 3*d.std()
Anomaly detection	Using the clusters to find strange points (far from every centroid).	Sort by the distance column.

Task 5.1 — One extreme customer

Add one extra row to X2: Income = 400, Spending = 99. Use K = 5 on Income and Spending.

1. Fit KMeans (n_init=10, random_state=SEED) without and with the extra customer.
2. Print the cluster sizes, the centroids (sorted by income) and the ARI between the two results on the original 200 customers.
3. Draw the two clusterings side by side.

Task 5.2 — Catch it and repair

Use the model fitted on the original 200 customers.

1. Compute every customer's distance to its nearest centroid, and the distance of the extra customer to the nearest centroid of the same model.
2. Apply the rule mean + 3 std (mean and std from the 200 real customers). How many real customers are flagged? Is the extra customer flagged?

3. Show the 5 real customers that are farthest from their centroid (CustomerID, Age, Income, Spending, Cluster, Distance).
4. Remove all flagged points, refit K = 5 and report the cluster sizes and the ARI against the original model on the 200 customers (use predict).

Questions to answer in Markdown (use your own numbers)

(a) What did the single extreme customer do to the clusters? Which real groups were affected? Use sizes and centroids.

(b) How well did the rule detect the extra customer? The rule also flagged some real customers: are they mistakes in the data or something worth looking at? Look at their Income and Spending.

(c) Name two ways to deal with outliers and one risk of simply deleting them.

Part 6 — From Clusters to Customer Segments (25 minutes)

Clusters are numbers (0, 1, 2 ...). The business needs names and actions. Cluster numbers are arbitrary labels: cluster 3 in one run has no meaning by itself. Use the original 200 customers and K = 5 on Income and Spending (random_state=SEED).

Key definitions

Term	What it means	How you write / read it
Cluster profile	The average of every feature inside each cluster, plus the cluster size. This is how you read what a cluster means.	df.groupby('Cluster').agg(...)
Segment	A cluster with a name and an action, for example 'High income / Low spending'.	df['Cluster'].map(names)
Crosstab	Counts or percentages of one category inside another (Gender inside Segment).	pd.crosstab(a, b, normalize='index')

Task 6.1 — Profile and name the segments

Make the clustering readable for a manager.

1. Add the cluster number to the data. Build a profile table: number of customers, mean Age, mean Income, mean Spending and % female for each cluster, sorted by Income.
2. Create segment names with a small rule (Income below 40 = Low, 70 or more = High, otherwise Mid; Spending below 35 = Low, 65 or more = High, otherwise Mid). Print the size of each segment.

Task 6.2 — Does Gender matter? Does Age matter?

Gender and Age were not used in the clusters. Check whether they would add something.

1. Show the % of women in each segment and the overall %.
2. Scale Age, Income and Spending together and print the average silhouette for K = 2 to 8.

Task 6.3 — A three-feature alternative

Cluster the scaled Age, Income and Spending with the best K from the table above.

1. Fit it, build its profile table (size, Age, Income, Spending) sorted by Age.
2. Print the ARI between this segmentation and your 2-feature K = 5 segmentation.

Questions to answer in Markdown (use your own numbers)

(a) Give each of the 5 segments a short business name and one concrete action (an offer or a message). Use the numbers from your profile table.

(b) Which segment is the most valuable now and which one is the biggest opportunity? Why?

(c) Does Gender help to separate the segments? Quote the percentages.

(d) Does adding Age change the story? Which segmentation would you present and why? (Careful: silhouette values from different feature sets are not directly comparable.)

Part 7 — Final Output and Wrap-Up (15 minutes)

Package your result like you would give it to a marketing team.

Key definitions

Term	What it means	How you write / read it
Deliverable	A file someone else can use without running your notebook.	df.to_csv('customer_segments.csv', index=False)

Task 7.1 — Save the segments

Create a CSV with CustomerID, Age, Gender, Income, Spending, Cluster and Segment.

1. Save it as customer_segments.csv and print the number of customers per segment.
2. Write a 6-8 line Executive Summary (Markdown): the K you chose and the evidence for it, the segments and their sizes, one action per segment (short), how you handled scaling and outliers, and one limitation.

Questions to answer in Markdown (use your own numbers)

(a) What was the most surprising result in this lab?

(b) If you had one more hour, what would you try next?

Time Allocation

Time	Activity
0-10 min	Part 0 — Load data, check, scatter plots
10-45 min	Part 1 — Lecture examples in code, own K-Means, compare with sklearn
45-75 min	Part 2 — Elbow, silhouette, silhouette by hand, practice problem
75-95 min	Part 3 — 30 seeds: starting points and stability
95-120 min	Part 4 — Scaling: unit change and the Wine data
120-140 min	Part 5 — Outlier experiment and distance rule
140-165 min	Part 6 — Profiles, segment names, gender/age checks
165-180 min	Part 7 — Output file and executive summary

Submission Requirements

Submit ONE completed notebook with all Parts in order, with Markdown headings that match the Part and Task numbers of this manual. At the top, include a Student Information block:

```
# LAB 10 — K-Means Clustering
## Student Information
Name:
Student ID:
Section:
GitHub:
Kaggle:
Dataset: Mall Customers (Kaggle) + Wine (UCI)
Dataset Source: <paste the URLs you used>
```

Required in the Notebook

- Every table, plot and question needs a written answer with your own numbers, directly below the code.
- Push the final notebook and customer_segments.csv to your GitHub repository.

Grading Rubric (20 Marks)

Criterion	Marks
Part 0 — Setup, data check, plots, written answers	1
Part 1 — Lecture examples reproduced, own K-Means function, comparison with sklearn	4
Part 2 — Elbow and silhouette for K=2..10, silhouette by hand, practice problem	4
Part 3 — 30-seed stability experiment and correct explanation	2
Part 4 — Scaling experiments (unit change + Wine) and correct interpretation	3
Part 5 — Outlier experiment, distance rule, repair	2
Part 6 — Profiles, segment names and actions, gender/age checks	3
Part 7 — Output file, executive summary, documentation quality	1
Total	**20**

Penalties

- Using KMeans() inside your own function in Task 1.3.
- Choosing K without showing both the elbow and the silhouette evidence.
- Clustering unscaled features of different units without comment (Part 4 shows why).
- Plots or tables without interpretation; copied answers; a notebook that does not run top to bottom; a missing Student Information block.